Kaggle Bot Architecture

Simulate10Next

Handover Presentation

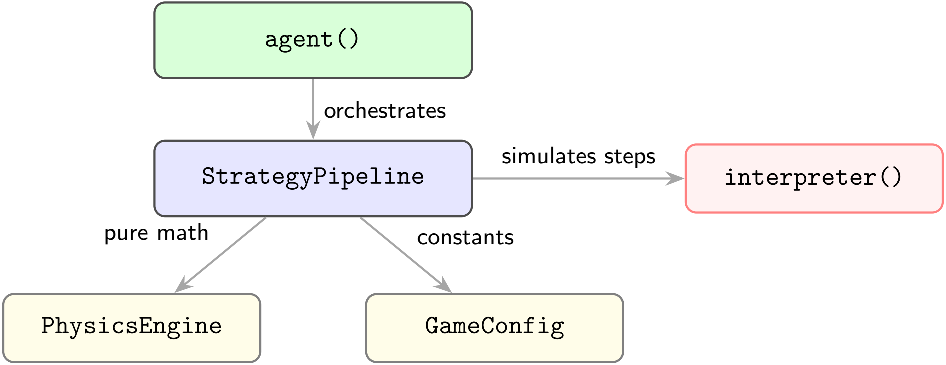
May 29, 2026

High-Level Philosophy

- **Data-Driven Design:** The bot handles the game state as Pandas DataFrames.
- **Vectorized Operations:**
  - Relies on DataFrame transformations rather than iterative loops for speed.
  - Converting to Polars Lazy would make it even faster.

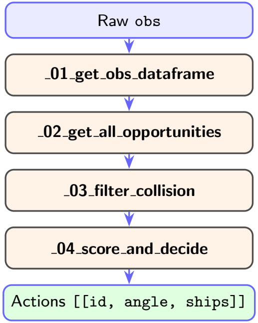
Presentation Overview

The Architecture



- `agent()` manages Kaggle I/O and global state (`step`, `player_id`).
- `StrategyPipeline` transforms each observation into moves via four sequential stages.
- `interpreter()` is the Kaggle game engine — called by `_01` to simulate 10 steps forward.
- `PhysicsEngine` and `GameConfig` are stateless and never modified.

The Pipeline Workflow



- **_01:** Simulates 10 future steps via `interpreter()`.
  - Outputs `df_s` (planet states) and `planet_disp`.
  - Labels each planet as `fix`, `moving`, or `comet`.
- **_02:** Cross-joins sources $\times$ targets across all steps.
  - Swept-pair collision finds valid intercept trajectories.
  - Outputs `pa` (Possible Attacks) with angle cones.
- **_03:** Removes occluded trajectories.
  - Drops any path blocked in the same angular cone by an earlier target.
- **_04:** Scores and selects one move per source.
  - Ranks by time-cost-adjusted production value.

Roadmap: Enhancing `_04_score_and_decide`

1. Role-Based Planet Logic (Supply Chains)

- **Conquerors** — Planets with ≥ 1 enemy within their 5 nearest neighbors.
  - Prioritized for frontline attacks.
- **Suppliers** — Planets where all 5 nearest neighbors are friendly.
  - Prioritized to send reinforcements to Conquerors.

2. Kinematic Targeting Profiles

- Scoring differentiates by motion-type pair (source $\times$ target): `fix`, `moving`, or `comet`.
- Moving targets are preferred — they allow attacking planets on the far side of the board by intercepting their orbit.

Long-Term Roadmap: Simulate10NextAction

Compute Caching (Beating the Time Limit)

- Pre-compute 500-step positions for all planets at $T = 0$.
- Cache Fix $\rightarrow$ Fix opportunities globally — geometry never changes.
- Cache Moving $\rightarrow$ Moving matrices; adapt each turn by rotating angles by $\omega \times \Delta t$.
- Track spawned fleets immediately — compute destination once, reuse every step.
- With caching in place: simulate all candidate actions 10 steps forward and select the subset that maximises the board score.